

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: Коммивояжёр. Динамическое программирование

Студент гр. 4345

Гавриш М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать точный и приближённый методы для решения задачи коммивояжера.

Задание

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

- n — количество городов ($5 \leq n \leq 15$).
- Матрица расстояний между городами размером $n \times n$, где $\text{graph}[i][j]$ обозначает расстояние от города i до города j . Если $\text{graph}[i][j]=0$ (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные:

- Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.
- Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.
- Если такого пути не существует, вывести "no path".

Вариант — 7

Точный метод: динамическое программирование (не МВиГ), рекурсивная реализация.

Приближённый алгоритм: АЛШ-1.

Требование перед сдачей: прохождение кода в задании 3.1 на Stepik.

Замечание к варианту 7. АЛШ-1 начинать со стартовой вершины.

Основные теоретические положения.

Задача коммивояжёра — это классическая комбинаторная задача, заключающаяся в поиске гамильтонова цикла минимального веса в полном взвешенном графе. Граф задаётся матрицей расстояний, где элемент A_{ij} обозначает стоимость перехода из города i в город j . Задача является NP-трудной, то время решения методом полного перебора составляет $O(n!)$, что делает его неприменимым даже для средних значений n .

Для решения задачи используются два основных класса алгоритмов:

1. Точные методы: Гарантируют нахождение оптимального решения. Наиболее эффективным для $n \leq 20$ является метод динамического программирования по подмножествам. Его сложность составляет $O(n^2 \cdot 2^n)$. Суть метода заключается в разбиении задачи на подзадачи.
2. Приближённые методы: Не гарантируют оптимальности, но работают значительно быстрее. Одним из простейших является Алгоритм лучшего шага (ALSH-1) — жадный алгоритм, который на каждом шаге выбирает ребро минимального веса, ведущее в ещё не посещённую вершину.

Выполнение работы

Для выполнения лабораторной работы были разработаны две программы на языке C++.

1. Генерация входных данных

Для создания тестовых матриц был написан файл `lb2_generator.cpp`. Программа запрашивает количество городов n и имя файла. В цикле заполняется матрица смежности:

- Диагональные элементы A_{ii} устанавливаются в 0.
- Остальные элементы генерируются случайно в диапазоне $[1, 50]$ с вероятностью 80%, либо 0 (отсутствие пути) с вероятностью 20%.

2. Решение задачи (Файл `lb2_solver.cpp`)

Программа считывает матрицу из файла или стандартного потока ввода и запускает два алгоритма.

А) Приближённый метод (АЛШ-1)

Реализован в функции `alsh1`. Алгоритм начинает с вершины 0. В цикле (пока не посещены все вершины) просматриваются все соседи текущей вершины. Выбирается та вершина v , для которой вес ребра `graph[current][v]` минимален и вершина ещё не посещена. Если на каком-то этапе доступных рёбер нет, алгоритм возвращает "no path". В конце проверяется возможность возврата в стартовую вершину.

Б) Точный метод (Динамическое программирование)

Реализован рекурсивно в функции `tsp_dp(mask, u)`.

- Состояние: Определяется парой $(mask, u)$, где `mask` — битовая маска посещённых вершин, u — текущая вершина.
- База рекурсии: Если `mask` заполнен всеми единицами (все города посещены), функция возвращает вес ребра `graph[u][0]` (возврат домой).
- Шаг рекурсии: Перебираются все вершины v , которых нет в маске. Вычисляется стоимость перехода в v плюс рекурсивный вызов для нового состояния. Выбирается минимум.

- Мемоизация: Результаты вычислений сохраняются в таблице memo, чтобы избежать повторных вычислений для одинаковых состояний.
- Восстановление пути: Параллельно с вычислением стоимости в массив next_node записывается индекс следующей вершины, давшей минимум. После вычисления общей стоимости путь восстанавливается проходом по этому массиву.

Описание ключевых переменных

В программе `lb2_solver.cpp` используются следующие конструкции:

1. `const int INF = 1000000000` — константа, обозначающая «бесконечность» — значение, которое заведомо больше любого возможного ответа. Используется для маркировки недостижимых состояний в матрице.
2. `vector<vector<int>> memo` — таблица мемоизации для ДП. `memo[mask][u]` хранит минимальную стоимость пути из вершины `u`, посещающего все вершины, отсутствующие в `mask`, и возвращающегося в `0`. Инициализируется значением `-1` (что означает "не вычислено").
3. `int mask` (Битовая маска) — целое число, используемое для кодирования подмножества посещённых вершин.
 - Операция `mask & (1 << v)` проверяет, посещена ли вершина `v`.
 - Операция `mask | (1 << v)` добавляет вершину `v` в множество посещённых.
 - Условие `mask == (1 << n) - 1` проверяет, посещены ли все вершины (все биты равны 1).
4. `vector<vector<int>> next_node` — массив для восстановления пути. `next_node[mask][u]` хранит индекс вершины `v`, переход в которую из `u` привёл к оптимальному решению для данного состояния.
5. `pair<int, vector<int>>` — структура данных, возвращаемая функцией АЛШ-1. Содержит пару: общую стоимость маршрута и вектор с последовательностью вершин.

Затраты алгоритмов по времени и памяти:

АЛШ-1: $O(n^2)$ по времени, $O(n)$ по памяти для хранения матриц.

ДП: $O(n^2 * 2^n)$ по времени, $O(n * 2^n)$ по памяти для хранения матриц.

Исходный код программ см. в приложении А.

Результаты тестирования программ см. в таблице 1 и 2.

Таблица 1 – Тестирование приближённого метода (АЛШ-1):

Ввод	Результат	Комментарий
5 0 1 13 23 7 12 0 15 18 28 21 29 0 33 28 23 19 34 0 38 5 40 7 39 0	106 0 1 2 4 3 0	Пример из условия задачи
3 0 1 0 1 0 1 0 1 0	no path	Пример из условия задачи
4 0 15 0 30 9 0 30 37 18 1 0 26 0 0 21 0	no path	Сгенерированная матрица

Таблица 2 – Тестирование точного метода:

Ввод	Результат	Комментарий
5 0 1 13 23 7 12 0 15 18 28 21 29 0 33 28 23 19 34 0 38 5 40 7 39 0	78 0 4 2 3 1 0	Пример из условия задачи
3 0 1 0 1 0 1 0 1 0	no path	Пример из условия задачи

4 0 15 0 30 9 0 30 37 18 1 0 26 0 0 21 0	61 0 3 2 1 0	Сгенерированная матрица
---	------------------------------------	---

Выводы

По алгоритму АЛШ-1 (Приближённый)

Алгоритм лучшего шага имеет высокую скорость работы, независящую от плотности графа. Его временная сложность составляет $O(n^2)$. Однако, как показали тесты, алгоритм обладает существенным недостатком: он принимает локально оптимальные решения, не заглядывая вперёд. В ориентированных графах с разрывами (нулями в матрице) это часто приводит к тому, что алгоритм заходит в вершину-тупик, из которой нет пути к оставшимся непосещённым городам или обратно в старт. В таких случаях АЛШ-1 выдаёт "no path", даже если глобальный маршрут существует.

По алгоритму ДП (Точный)

Рекурсивное динамическое программирование гарантированно находит оптимальный маршрут минимальной стоимости, если он существует. Благодаря использованию мемоизации, алгоритм избегает экспоненциального роста числа вызовов, характерного для полного перебора. Недостатком метода является высокое потребление памяти $O(n \cdot 2^n)$ и время выполнения, которое растёт экспоненциально. Для $n > 20$ данный метод становится неэффективным из-за нехватки ресурсов.

В ходе работы была успешно реализована генерация тестов, сохранение данных в файлы и сравнительный анализ двух подходов к решению NP-трудной задачи.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл lb2_solve.cpp:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <sstream>
#include <climits>
#include <algorithm>
#include <bitset>

using namespace std;

const int INF = 1000000000;
int n;
vector<vector<int>>> graph;

string mask_to_binary(int mask, int bits) {
    return bitset<15>(mask).to_string().substr(15 - bits);
}

pair<int, vector<int>>> alsh1() {
    cout << "\n=== ЗАПУСК АЛШ-1 ===" << endl;

    vector<bool> visited(n, false);
    int current = 0;
    visited[current] = true;
    vector<int> path = {current};
    int total_cost = 0;

    cout << "Старт в городе 0" << endl;

    for (int iter = 0; iter < n - 1; ++iter) {
        int best_next = -1;
        int min_dist = INF;

        cout << "\n[Шаг " << (iter + 1) << "] Текущий город: " <<
current
        << ", посещено: ";
        for (int i = 0; i < n; ++i)
            if (visited[i]) cout << i << " ";
        cout << endl;
        cout << " Перебор кандидатов:" << endl;

        for (int v = 0; v < n; ++v) {
            if (!visited[v] && graph[current][v] > 0) {
                cout << "    Город " << v << ": расстояние = " <<
graph[current][v];
                if (graph[current][v] < min_dist) {
                    min_dist = graph[current][v];
                    best_next = v;
                    cout << " - НОВЫЙ МИНИМУМ!" << endl;
                } else {
                    cout << endl;
                }
            }
        }
    }
}
```

```

    }
}

if (best_next == -1) {
    cout << " Нет доступных городов - тупик!" << endl;
    return {INF, {}};
}

cout << " Выбран город " << best_next << " (расстояние " <<
min_dist << ")" << endl;

visited[best_next] = true;
path.push_back(best_next);
total_cost += min_dist;
current = best_next;

cout << " Путь: ";
for (size_t i = 0; i < path.size(); ++i) {
    if (i > 0) cout << " -> ";
    cout << path[i];
}
cout << ", общая стоимость: " << total_cost << endl;
}

cout << "\nФиниш. Попытка вернуться в город 0..." << endl;
if (graph[current][0] > 0) {
    total_cost += graph[current][0];
    path.push_back(0);
    cout << " Успех! Ребро " << current << " -> 0 (вес " <<
graph[current][0] << ")" << endl;
    cout << " Итоговый путь: ";
    for (size_t i = 0; i < path.size(); ++i) {
        if (i > 0) cout << " -> ";
        cout << path[i];
    }
    cout << ", общая стоимость: " << total_cost << endl;
    return {total_cost, path};
}
cout << " Нет пути из " << current << " в 0" << endl;
return {INF, {}};
}

vector<vector<int>>> memo;
vector<vector<int>>> next_node;

int tsp_dp(int mask, int u, int depth = 0) {
    string indent(depth * 2, ' ');

    cout << indent << "Рекурсия. tsp_dp(mask=" << mask_to_binary(mask,
n)
        << ", u=" << u << ")" << endl;

    if (mask == (1 << n) - 1) {
        cout << indent << " Итог. Все города посещены. Ребро " << u
<< " -> 0: ";
        if (graph[u][0] > 0) {
            cout << graph[u][0] << endl;
        } else {

```

```

        cout << "НЕТ ПУТИ (0)" << endl;
    }
    return (graph[u][0] > 0) ? graph[u][0] : INF;
}

if (memo[mask][u] != -1) {
    cout << indent << "    МЕМО. Возвращаем кэшированное значение: "
<< memo[mask][u] << endl;
    return memo[mask][u];
}

int ans = INF;
int best_nxt = -1;

cout << indent << "    Перебор кандидатов из города " << u << ":" <<
endl;

for (int v = 0; v < n; ++v) {
    if (!(mask & (1 << v)) && graph[u][v] > 0) {
        cout << indent << "        Кандидат " << v << " (ребро " << u
<< "->" << v
            << " = " << graph[u][v] << ")..." << endl;

        int res = graph[u][v] + tsp_dp(mask | (1 << v), v);

        cout << indent << "        Результат для кандидата " << v <<
": ";

        if (res >= INF) {
            cout << "INF (тупик)" << endl;
        } else {
            cout << graph[u][v] << " + " << (res - graph[u][v]) <<
" = " << res << endl;
        }

        if (res < ans) {
            ans = res;
            best_nxt = v;
            cout << indent << "            НОВЫЙ МИНИМУМ! ans = " << ans
<< ", best_nxt = " << v << endl;
        }
    }
}

memo[mask][u] = ans;
next_node[mask][u] = best_nxt;

cout << indent << "    Сохранение в МЕМО. мемо[" <<
mask_to_binary(mask, n) << "][" << u << "] = ";
if (ans >= INF) {
    cout << "INF" << endl;
} else {
    cout << ans << " (следующий: " << best_nxt << ")" << endl;
}

return ans;
}

void solve_exact() {

```

```

    cout << "\n=== ЗАПУСК ДП (Точный метод) ===" << endl;
    cout << "Инициализация таблиц: " << (1 << n) << " масок × " << n
    << " вершин" << endl;

    memo.assign(1 << n, vector<int>(n, -1));
    next_node.assign(1 << n, vector<int>(n, -1));

    cout << "\nВызов tsp_dp(mask=" << mask_to_binary(1, n) << ", u=0)"
    << endl;
    cout << string(50, '-') << endl;

    int min_cost = tsp_dp(1, 0);

    cout << string(50, '-') << endl;
    cout << "Минимальная стоимость: ";
    if (min_cost >= INF) {
        cout << "INF (no path)" << endl;
    } else {
        cout << min_cost << endl;
        cout << "\nВосстановление пути:" << endl;

        vector<int> path = {0};
        int mask = 1;
        int u = 0;

        while (mask != (1 << n) - 1) {
            int v = next_node[mask][u];
            cout << "    mask=" << mask_to_binary(mask, n) << ", u=" <<
u
                << " следующий: " << v << endl;
            path.push_back(v);
            mask |= (1 << v);
            u = v;
        }
        path.push_back(0);

        cout << "    Финальный путь: ";
        for (size_t i = 0; i < path.size(); ++i) {
            if (i > 0) cout << " -> ";
            cout << path[i];
        }
        cout << endl;

        for (size_t i = 0; i < path.size(); ++i) {
            if (i > 0) cout << " ";
            cout << path[i];
        }
        cout << endl;
    }
}

int main(int argc, char* argv[]) {
    if (argc > 1) {
        ifstream file(argv[1]);
        if (!file.is_open()) {
            cerr << "Cannot open file: " << argv[1] << endl;
            return 1;
        }
    }
}

```

```

        file >> n;
        graph.assign(n, vector<int>(n));
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                file >> graph[i][j];
            }
        }
        file.close();
    } else {
        cin >> n;
        graph.assign(n, vector<int>(n));
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                cin >> graph[i][j];
            }
        }
    }
}

pair<int, vector<int>> approx_result = alsh1();
int approx_cost = approx_result.first;
vector<int> approx_path = approx_result.second;

if (approx_cost == INF) {
    cout << "no path" << endl;
} else {
    cout << approx_cost << endl;
    for (size_t i = 0; i < approx_path.size(); ++i) {
        if (i > 0) cout << " ";
        cout << approx_path[i];
    }
    cout << endl;
}
cout << endl;

solve_exact();

return 0;
}

```

Файл lb2_generator:

```

#include <iostream>
#include <fstream>
#include <random>
#include <chrono>

using namespace std;

int main() {
    int n;
    cout << "Количество городов: ";
    cin >> n;

    cin.ignore();

    string filename;
    cout << "Имя файла (Enter для input.txt): ";
    getline(cin, filename);
}

```

```

    if (filename.empty()) {
        filename = "input.txt";
    }

    unsigned seed =
chrono::system_clock::now().time_since_epoch().count();
    mt19937 gen(seed);

    uniform_int_distribution<int> weight_dist(1, 50);
    uniform_real_distribution<double> edge_prob(0.0, 1.0);

    ofstream file(filename);
    if (!file.is_open()) {
        cerr << "Ошибка: не удалось создать файл " << filename <<
endl;
        return 1;
    }

    file << n << "\n";
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (i == j) {
                file << 0;
            } else if (edge_prob(gen) > 0.2) {
                file << weight_dist(gen);
            } else {
                file << 0;
            }
            if (j < n - 1) file << " ";
        }
        file << "\n";
    }

    file.close();
    cout << "Матрица сохранена в: " << filename << endl;
    return 0;
}

```